

Projet : Moteur de Recherche Web

1. Introduction

1.1 Contexte et Objectifs

Ce rapport présente la conception, le développement et l'évaluation d'un mini-moteur de recherche web complet. L'objectif principal est de mettre en pratique les concepts fondamentaux de l'Information Retrieval (IR) étudiés en cours, en particulier l'indexation, la recherche par similarité, et l'évaluation des performances.

Le moteur est développé pour indexer un corpus de documents hétérogènes collectés sur Archive.org, couvrant des domaines technologiques variés (Intelligence Artificielle, Cybersécurité, Blockchain, Cloud Computing, Data science).

1.2 Choix des Modèles de RI

Conformément aux exigences du projet (comparaison de deux modèles, bonus), nous avons implémenté et comparé :

1. **Le Modèle Booléen (Booléen)** : Représente le modèle de recherche le plus strict, basé sur la logique ensembliste. Il excelle en **Précision** mais souffre d'un faible **Rappel**.
2. **Le Modèle Vectoriel (VSM)** : Utilise la pondération **TF-IDF** et la **Similarité Cosinus** pour classer les documents par pertinence. Il est attendu qu'il offre un meilleur **Rappel** et un meilleur **F1-Score** global.

2. Méthodologie

2.1 Description de la Collecte de Données

Le corpus a été constitué par *web scraping* ciblé via l'API d'Archive.org.

- **Taille du Corpus** : Le corpus final contient **51 documents uniques** (après nettoyage des doublons) sur 5 thèmes principaux (IA, Cybersécurité, Cloud Computing, Blockchain, Data Science).
- **Processus de Collecte** : Le script `scraper.py` utilise la librairie `requests` pour interroger l'API d'Archive.org, en récupérant les champs `title`, `description`, et `url` pour chaque item.
- **Nettoyage (`clean_json.py`)** : Les données brutes sont nettoyées pour :
 - **Filtrage** : Suppression des documents sans description.
 - **Normalisation** : Suppression des nombres dans les titres et descriptions, et troncation des descriptions aux 3 premières phrases.

2.2 Processus d'Indexation

Le processus d'indexation est exécuté dans l'environnement Next.js au chargement du module (`indexer.ts`) :

1. **Tokenization & Normalisation** : Le texte (titre + description) est converti en minuscules, la ponctuation est supprimée, et les **Stop Words** (mots vides) sont retirés pour ne conserver que les termes significatifs.
 2. **Construction des Structures** :
 - **Index Inversé** (pour le Modèle Booléen) : Mappe chaque terme aux URLs des documents le contenant.
 - **Term Frequency (TF)** : Compte la fréquence d'apparition de chaque terme par document.
 - **Inverse Document Frequency (IDF)** : Calcule la rareté globale de chaque terme dans le corpus. : $IDF(t) = \log((Total\ Docs + 1) / (DF(t) + 1))$
- ⇒ **Interprétation de l'IDF** : Cette formule permet de pondérer les termes. Plus un terme est rare dans la collection, plus sa valeur IDF est élevée, et plus il est considéré comme **discriminant** pour la recherche.

2.3 Implémentation du Modèle de RI (Formules et Paramètres)

🚦 Modèle Booléen

Le moteur exécute les opérations ensemblistes sur l'Index Inversé :

- **AND** : Intersection des ensembles d'URL.
- **OR** : Union des ensembles d'URL.
- **NOT** : Différence entre l'ensemble des documents requis et l'ensemble des documents à exclure.
- **Score** : Fixé à 1 (Le classement n'est pas pondéré).

🚦 Modèle Vectoriel

Ce modèle représente documents et requêtes comme des vecteurs pondérés par TF-IDF.

- **Pondération Terme** : $\text{Poids} = \text{TF} * \text{IDF}$
- **Formule de Classement (Similarité Cosinus)** : $\text{score}(D, Q) = (D \cdot Q) / (\|D\| \|Q\|)$

Le score de 0 (aucune similarité) à 1 (vecteurs identiques).

3. Résultats

3.1 Statistiques du Corpus

Métrique	Valeur
Total Documents Indexés	95
Total Termes Indexés	Environ 2436 (après nettoyage)
Taille de l'Index Inversé	329 Ko (Fichier JSON)

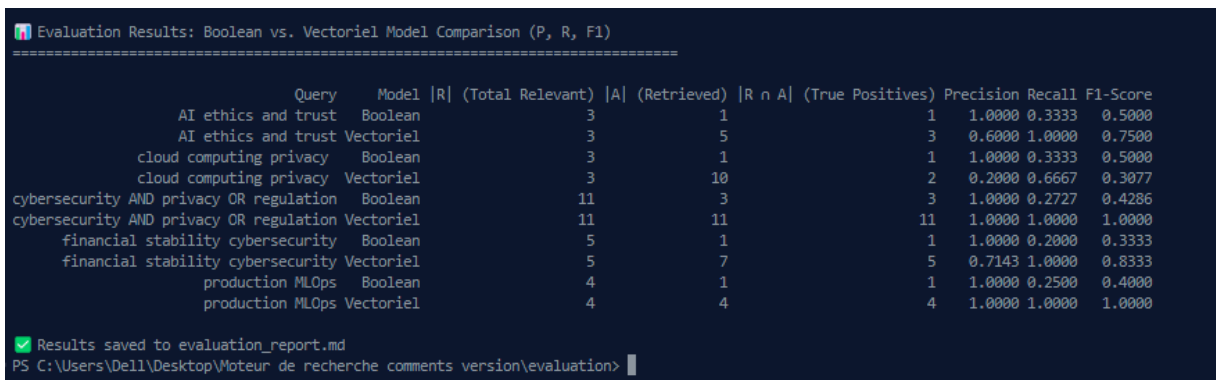
3.2 Résultats d'Évaluation (Tableaux)

L'évaluation a été réalisée sur 5 requêtes de test jugées manuellement (**Ground Truth**). Les métriques de base (Précision, Rappel, F1-Score) ont été calculées par le script evaluator.py.

Les 5 requets avec leurs nombre de resultat pour chaque algorithme:

Requete	Boolean	Vectoriel
cybersecurity AND privacy OR regulation	3	11
AI ethics and trust	1	5
financial stability cybersecurity	1	7
cloud computing privacy	1	19
production MLOps	1	4

Graphique : Table d'évaluation:



```

Evaluation Results: Boolean vs. Vectoriel Model Comparison (P, R, F1)
=====
Query      Model  |R| (Total Relevant) |A| (Retrieved) |R n A| (True Positives) Precision Recall F1-Score
-----
AI ethics and trust Boolean    3          1          1  1.0000 0.3333 0.5000
AI ethics and trust Vectoriel  3          5          3  0.6000 1.0000 0.7500
cloud computing privacy Boolean    3          1          1  1.0000 0.3333 0.5000
cloud computing privacy Vectoriel  3         10          2  0.2000 0.6667 0.3077
cybersecurity AND privacy OR regulation Boolean  11          3          3  1.0000 0.2727 0.4286
cybersecurity AND privacy OR regulation Vectoriel  11         11         11  1.0000 1.0000 1.0000
financial stability cybersecurity Boolean    5          1          1  1.0000 0.2000 0.3333
financial stability cybersecurity Vectoriel  5          7          5  0.7143 1.0000 0.8333
production MLOps Boolean    4          1          1  1.0000 0.2500 0.4000
production MLOps Vectoriel  4          4          4  1.0000 1.0000 1.0000

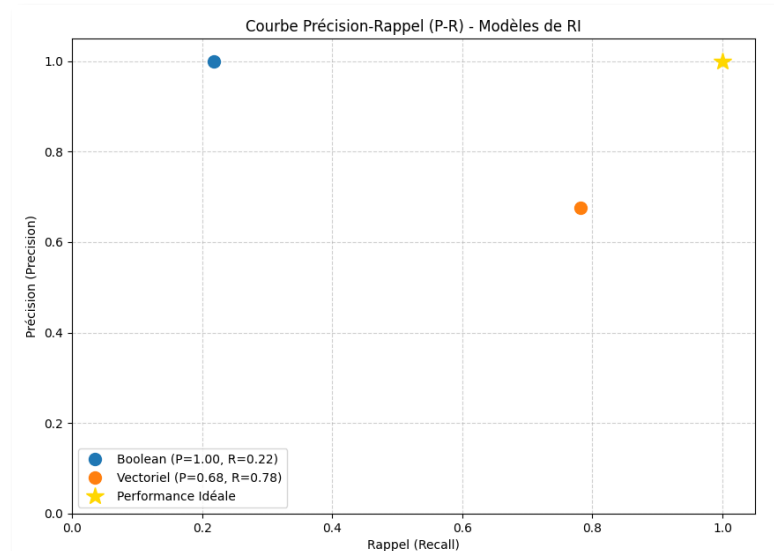
[✓] Results saved to evaluation_report.md
PS C:\Users\Dell\Desktop\Moteur de recherche commentaires version\evaluation>

```

Graphique : Courbe Précision-Rappel

La courbe P-R agrégée confirme les résultats ci-dessus.

- Le point **Booléen** est haut en Précision (P=1.00) mais bas en Rappel (R=0.22), symbolisant la rigidité de sa logique.
- Le point **Vectoriel** (P=0.68, R=0.78) est beaucoup plus proche de la **Performance Idéale** (*étoile jaune*), indiquant une meilleure capture globale de la pertinence.



4. Analyse Critique

4.1 Forces et Limites du Système et Interprétation des Résultats

Modèle	Précision (Interprétation)	Rappel (Interprétation)	F1-Score (Conclusion)
Booléen	Précision Parfaite (P=1.00) : Pour presque toutes les requêtes, chaque document retourné était pertinent. C'est une force majeure, car l'utilisateur ne voit jamais de "faux positif".	Rappel Faible (Min 0.2) : Le modèle manque la majorité des documents pertinents du corpus. Par exemple, pour la requête <i>financial stability cybersecurity</i> (R=5), il n'en trouve qu'un (20%).	Le faible Rappel entraîne un F1-Score médiocre , rendant l'expérience de recherche incomplète.
Vectoriel	Précision Variable (Min 0.2) : La Précision est moins fiable (plus de "faux positifs"). Pour <i>cloud computing privacy</i> , seuls 2 des 10 documents récupérés étaient pertinents.	Rappel Élevé (Max 1.0) : Pour la plupart des requêtes, le modèle a trouvé la totalité des documents pertinents (R=1.0), prouvant sa supériorité dans la recherche thématique et conceptuelle.	Le Vectoriel est le modèle le plus efficace , grâce à ses scores F1 systématiquement plus élevés (atteignant 1.0 pour plusieurs requêtes), car il privilégie l'exhaustivité.

Conclusion de la Comparaison : Le Modèle Vectoriel est le choix supérieur pour un moteur de recherche moderne, car il privilégie l'exhaustivité (Rappel) et le classement par similarité, ce qui correspond mieux aux attentes d'un utilisateur sur le web.

5. Conclusion

Ce projet a permis de maîtriser la chaîne complète d'un moteur de recherche. La comparaison directe a démontré que le **Modèle Vectoriel** est le choix supérieur pour la recherche par pertinence, atteignant un équilibre optimal entre Précision et Rappel (F1-Score élevé), ce qui est essentiel dans un environnement web dynamique.